Cluster Computing (2025) 28:1014
https://doi.org/10.1007/s10586-025-05725-y

Ensemble multi-label machine learning solidity smart contract vulnerability detection model

Rashed Alnuman[1] · Tayyab Sajid[1] · Wesam Almobaideen[1,2] · Qusai Hasan[1]

Received: 12 March 2025 / Revised: 3 July 2025 / Accepted: 3 August 2025 / Published online: 17 October 2025
© The Author(s) 2025

Abstract
Blockchain technology has revolutionized digital financial transactions and asset ownership by enabling decentralized and automated operations through smart contracts. Solidity smart contracts, used in the Ethereum blockchain network, facilitate secure and trustless execution of agreements. However, like any code, smart contracts are prone to vulnerabilities. Considering the assets and value of currency these smart contracts handle, their exploitation leads to severe financial losses and loss of operations. Such exploits have resulted in billions of dollars in stolen or locked assets. In this paper, we present an ensemble multilabel classifier model approach for the automated detection of vulnerabilities in Solidity smart contracts using a real smart contract dataset, with a detailed methodological process that includes processing the dataset. The proposed model stack achieves excellent results with F1 scores ranging from 82.0% to 99.9% for each vulnerability dataset. The proposed model is also compared with common static analyzer tools and models proposed in the literature following a similar approach. Moreover, we package the models into a web application, demonstrating deployment and functionality.

Keywords Smart contract · Blockchain · Solidity · Vulnerability detection · Etherlock · Reentrancy · Integer underflow/Overflow · Block dependency

1 Introduction

Blockchain is an immutable distributed peer-to-peer ledger used to securely store records of transactions [1]. Blockchain was designed for use in financial applications, mainly the use of cryptocurrencies for secure and anonymous transactions. The key defining characteristics include the ability of ensuring the integrity and availability of the data on it. This feature is supported by the use of cryptographic hashing where each block contains a cryptographic hash of the previous block along with timestamp and transaction data referred to as transactional data (TX) [2]. As such, changing of information is extremely difficult and essentially not possible under normal conditions. This allows blockchain to be a suitable technology for financial services, supply chains [3], IoT [4], healthcare [5, 6], and much more.

Blockchains also employ various mechanisms to support its operations. The use of consensus algorithms particularly is a critical aspect of blockchain. Consensus algorithms allow for participants of the blockchain to agree on the validity of the transactions in a specific way, eliminating the requirement of a central authority [7]. This ensures that each new block added to the chain is the only version of truth accepted by all legitimate participants. Thus, maintaining the integrity of the blockchain. Most common deployed consensus algorithms include: Proof of Work (PoW) which is used by Bitcoin, Proof of Stake, Delegated Proof of Stake (DPoS), Practical Byzantine Fault Tolerance (PBFT) and Proof of Authority (PoA) [8].

✉ Rashed Alnuman
rha4365@rit.edu

Tayyab Sajid
ts3138@rit.edu

Wesam Almobaideen
wxacad@rit.edu; wesmoba@ju.edu.jo

Qusai Hasan
qxhcad@rit.edu

[1] Department of Electrical Engineering and Computing Sciences, Rochester Institute of Technology Dubai, Dubai, United Arab Emirates

[2] Department of Computer Science, The University of Jordan, Amman, Jordan

Smart contracts are another crucial aspect of blockchain technology. Smart contracts are self-executing if/when then logical statements that perform functions and actions based on a set of met prerequisites and conditions. Smart Contracts eliminate the requirement for manual human intervention in transactions and facilitate the process through automation [7]. Asymmetric cryptography employed in smart contract ensures data security, confidentiality and party authentication ensure secure and verifiable interaction between the transacting parties. All this, and the fact that deployed smart contracts are immutable, foster trust in the blockchain network. While the nature of the smart contract provides extensive security, its implementation may not be. Languages such as Solidity and Vyper for smart contracts for Ethereum and TEAL for Algorand, suffer from code-level vulnerabilities and insecure logic can lead to critical vulnerabilities. While smart contracts provide security to the assets on the blockchain, security must be provided to the smart contract itself as well.

Damages caused due to insecure smart contracts is extensive and is still an ongoing problem. The nature of assets and crypto currency value means the financial damages are severe. If a contract transferring such large amounts is exploited, such that the funds are stolen or withheld, then the effects can be devastating. Previously, cybercriminals stole $613 Million Dollars' worth of crypto currency by exploiting the Polynetwork smart contract that transfers funds between blockchains [9]. Moreover, between 2022 and start of 2025 atleast over $6.9 Billion dollars USD worth of crypto currencies through exploiting smart contracts [10] with the biggest and most incident being the Bybit hack. Attackers were able to exploit a vulnerability in the cold storage contract, allowing the theft of aproximately $1.46 Billion Dollars USD [11]. Such calamitous damage not only drives a serious need to secure smart contracts at a production stage, but also tools to effectively identify vulnerable contracts already deployed on the blockchain. This dangerous problem drives the motivation of our work, to contribute to the literature by creating a machine learning based auditing tool to identify Etherlock attacks.

In this research, we build upon a previous published study [12] which introduced a SVM machine learning model for detection of Etherlock vulnerabilities using real smart contracts. This extension of the work proposes a ensemble model integrating four stacked models, providing detection of other vulnerabilities. The contributions of this work are as follows.

- Providing a high accuracy machine learning ensemble model for Solidity smart contract vulnerability detection.
- Provide an in-depth discussion of the methodology of converting real smart contracts from the blockchain network into a processable labeled dataset.
- Provide a model stack for the detection for Reentrancy, Etherlock, Integer Overflow/Underflow, and Block Dependency.
- Comparison of machine learning performance against static analyzers, and comparison of the proposed model against most similiar works in the literature.

The rest of this paper is organized as follows: Sect. 2 provides background information, covering smart contract vulnerabilities and machine learning algorithms. Section 3 presents a literature review of recent related research with an analysis. Section 4 details contract labeling process, contract preprocessing, and dataset description. Section 5 introduces the proposed model and its results with comprehensive analysis, comparisons, limitations and future works. Finally, Sect. 6 concludes the study.

2 Background

Smart contracts are self-executing pieces of code that run on the blockchain, enabling a wide range of applications such as decentralized finance (DeFi), supply chain monitoring, and identity management. Because they rely on consensus across nodes in the network, smart contracts are considered transparent and (once deployed) typically immutable. This immutability can be advantageous for ensuring trust, but also raises unique security challenges. Since smart contracts cannot be modified, only destroyed with a kill switch [13], security flaws are particularly dangerous [14].

In recent years, machine learning (ML) techniques have also grown in popularity for tasks such as data classification, anomaly detection, and automated decision-making. ML algorithms learn from data and identify patterns that can help in predicting outcomes or detecting irregularities. Within the blockchain and smart contract domain, ML may be employed to detect suspicious transactions, classify

contract behavior, or identify potential vulnerabilities before they can be exploited.

In the following subsections, we examine both common smart contract vulnerabilities and the fundamentals of machine learning approaches that can be applied to mitigate or detect these issues.

2.1 Smart contract vulnerabilities

Common vulnerabilities include reentrancy attacks, integer overflows/underflows, arbitrary memory access, bad randomness or block dependancy among other logical vulnerabilities. Exploiting these weaknesses can result in loss of funds, contract manipulation, or destructive unintended behavior.

2.1.1 Arbitrary memory access

Arbitrary Memory Access is a vulnerability in Solidity smart contracts that allows attackers to read or write memory outside intended boundaries, potentially leading to data leakage, unauthorized actions, or state corruption.

```
pragma solidity ^0.8.0;

contract VulnerableMemoryAccess {
  bytes public data;

  function writeData(uint256 index, bytes32 value) public {
      assembly {
          let ptr := add(data.slot, index) // Potentially unsafe memory access
          sstore(ptr, value) // Storing value at an arbitrary memory location
      }
  }

  function readData(uint256 index) public view returns
  (bytes32) {
      bytes32 result;
      assembly {
        let ptr := add(data.slot, index) // Potentially unsafe memory access
        result := sload(ptr) // Reading from an arbitrary memory location
      }
      return result;
    }
}
```

This often arises from incorrect memory handling, such as improper use of low-level assembly (mload, mstore, calldataload) or unsafe indexing in storage operations. For instance, in a vulnerable contract, an attacker could pass a large index to access unintended storage slots, potentially modifying critical contract variables. A common exploit scenario involves delegate calls to untrusted contracts, which can manipulate storage unpredictably [15]. To mitigate this, developers should use Solidity's built-in storage mechanisms, validate index bounds, avoid unnecessary use of assembly, and implement strict access controls. Below is an example demonstrating the vulnerability:

An attacker can pass a large index value to access or modify unintended storage slots. This may allow reading/modifying critical contract variables or other storage slots in the contract.

2.1.2 Assertion failure

An assertion failure in Solidity occurs when an assert() statement evaluates to false, causing the contract to revert and consume all remaining gas. Unlike require(), which is used for input validation and expected errors, assert() is typically reserved for enforcing invariants-conditions that should never be false under normal execution. If an assertion fails, it often indicates a critical bug in the contract logic, such as arithmetic overflows (in versions before Solidity 0.8.0), incorrect assumptions about state changes, or unintended modifications in storage variables [16]. Since assert() failures represent unexpected errors, they highlight fundamental flaws in the contract that may compromise security and functionality. Below is an example illustrating an assertion failure scenario where a contract incorrectly assumes that a counter will always be positive.

```
pragma solidity ^0.8.0;

contract AssertionFailureExample {
    uint256 public counter;

    function decrement(uint256 value) public {
        // Ensuring counter never goes below zero
        assert(counter >= value);
        counter -= value;
    }
}
```

In the above example, if decrement() is called with a value greater than counter, the assertion will fail, causing the transaction to revert. Proper use of require() would be more appropriate in this case, as it prevents user errors rather than detecting internal contract bugs.

2.1.3 Etherlock

The EtherLock vulnerability occurs when a smart contract locks Ether without providing a mechanism to withdraw or transfer the funds. A common cause of this issue is improper or missing access control, which can restrict authorized users from executing withdrawal functions. In some cases, the contract may lack a withdrawal function entirely, or contain logic flaws that unintentionally block fund transfers. Since smart contracts are immutable once deployed, any Ether sent to a contract affected by this vulnerability may become permanently inaccessible. This vulnerability is especially critical in fundraising, escrow, or vault-like contracts, where controlled and authorized fund retrieval is essential.

Attackers can particularly exploit smart contracts that store large amounts of Ether, such as vaults or cold storage contracts. Without proper withdrawal mechanisms, these funds can become locked, rendering them inaccessible [12]. For instance, in 2017, a vulnerability in the Parity Wallet's multisignature contract allowed an unauthorized user to become the contract owner and subsequently destroy it, leading to over $150 million worth of Ether being frozen [17].

The above example demonstrates Incorrect balance deduction before the transfer. The contract deducts the balance before confirming that the transfer was successful.If the transfer fails (e.g., due to a gas limit issue or the recipient being a smart contract that reverts), the Ether remains in the contract while the user's balance is already reduced. The user can no longer withdraw the locked funds since their balance is incorrectly set to zero. Without a failsafe mechanism or owner-controlled emergency withdrawal function, funds remain stuck in the contract permanently if transfers continue failing.

2.1.4 Block dependency

Block dependency occurs when a smart contract relies on block-specific variables (such as block.number, block.timestamp, or block.difficulty) for critical decision-making. Since these values can be influenced or predicted by miners or validators to some extent, attackers may exploit this dependency to manipulate contract behavior.

```
pragma solidity ^0.8.0;

contract LockedFunds {
  mapping(address => uint256) public balances;

  function deposit() external payable {
    require(msg.value > 0, "Must deposit Ether");
    balances[msg.sender] += msg.value;
  }

  function withdraw(uint256 amount) external {
    require(amount > 0, "Amount must be greater than zero");
    require(balances[msg.sender] >= amount, "Insufficient balance");

    // Logic Flaw: Incorrectly resetting the balance before transfer
    balances[msg.sender] -= amount;

    (bool success, ) = msg.sender.call{value: amount}("");
    require(success, "Transfer failed");
  }}
```

```
pragma solidity ^0.8.0;

contract BlockDependentLottery {
    address public winner;
    uint256 public lastDrawTime;

    function participate() external payable {
        require(msg.value == 1 ether, "Entry fee is 1 ETH");
    }
    function drawWinner() external {
        require(block.timestamp > lastDrawTime + 1 hours, "Wait the next draw");
        lastDrawTime = block.timestamp;

        // Vulnerable Randomness Generation
        uint256 winningNumber = uint256(keccak256( abi.encodePacked(block.timestamp
            , block.difficulty, msg.sender))) % 100;

        if (winningNumber == 42) {
            winner = msg.sender;
        }
    }}
```

Block dependency is particularly dangerous in applications like random number generation, time-based conditions, or reward distributions, where manipulation can lead to unfair advantages or financial loss [18]. The above example of a lottery is susceptible to miner manipulation. Since block.timestamp and block.difficulty are partially controlled by miners (especially on PoW networks), a miner could delay blocks until they find a favorable timestamp to win the lottery. An attacker can simulate future blocks off-chain, testing different addresses until they find a seed that results in winning. They can then send their transaction at the exact block that benefits them. If a user calls drawWinner() multiple times on a test network, they can predict when they will win and only participate when they have a high chance of success. In general, the reliance on block parameters is dangerous due to the information being dependent on miners and the fact that attackers have access to the information and are able to simulate it.

2.1.5 Integer Overflow/Underflow

Integer overflow and underflow occur when an arithmetic operation exceeds the storage limit of a variable type, causing unintended behavior. In Solidity versions prior to 0.8.0, integers would wrap around if they exceeded their maximum (overflow) or fell below zero (underflow). Attackers could exploit this flaw to manipulate contract logic, bypass restrictions, or drain funds. Since Solidity 0.8.0, overflows and underflows automatically revert transactions, but developers using older versions or assembly must still handle these risks manually [19].

```
pragma solidity ^0.8.0;

contract VulnerableCounter {
    uint8 public count = 255; //Maximum value for uint8

    function increment() public {
        count += 1; //Overflow occurs, wraps around to 0
    }

    function decrement() public {
        count -= 1; //Underflow occurs, wraps around to 255
    }
}
```

The increment() function increases count beyond its maximum (255 for uint8), causing it to wrap around to 0. If a contract relies on count for access control or limits, an attacker can reset it and bypass restrictions. Calling decrement() when count is 0 causes an underflow, making count wrap around to 255. This allows an attacker to set a value far higher than intended, breaking business logic.

2.1.6 Reentrancy

Reentrancy is a critical smart contract vulnerability where an external contract repeatedly calls back into the vulnerable contract before the first execution is complete, allowing the attacker to drain funds or manipulate contract state unexpectedly. This happens when a contract makes an external call to another contract before updating its state, allowing the attacker to exploit the unfinalized transaction and re-enter the function multiple times before the state changes [20].

```
pragma solidity ^0.8.0;

contract VulnerableBank {
    mapping(address => uint256) public balances;

    function deposit() external payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() external {
        uint256 amount = balances[msg.sender];
        require(amount > 0, "No funds to withdraw");

        // Vulnerable external call before state update
        (bool success, ) = msg.sender.call{value:amount}("");
        require(success, "Transfer failed");

        balances[msg.sender] = 0; // State update happens after the call
    }
}
```

In the above contract, the attacker deposits Ether into the contract. Deploys a malicious contract that overrides receive() or fallback() functions to call withdraw() recursively, where the vulnerable contract sends Ether before setting balances[msg.sender] = 0. Attacker is able re-enters the withdraw() function before the balance updates, withdrawing funds multiple times in a single transaction.

2.2 Machine learning algorithms background

Detecting vulnerabilities, attacks, and anomalies that indicate malicious activity is a core application of machine learning. Whether through anomaly detection, by establishing a baseline of normal system behavior, or by using labeled datasets derived from attack logs, machine learning provides an effective and foundational approach to securing computer systems. It enables advanced pattern recognition, allowing the differentiation between malicious and non-malicious actions or data. Machine learning techniques have been successfully applied to various cybersecurity domains, including network attack detection [21, 22], cybersecurity threat detection in Heating Ventilation and Air Conditioning (HVAC) systems [23], Detection and identification of malware [24, 25] or detection of code vulnerabilities through pattern analysis and semantic interpretation [26, 27].

2.2.1 Support vector machine

A supervised machine learning algorithm used in classification problems and regression tasks. It classifies two classes of data by attempting to apply an optimal hyperplane that best separates the data points of the two classes in a high-dimensional space. The hyperplane is chosen such that the margin representing the distance between the hyperplane and the closest data points is maximized. This is achieved through transforming input data into a higher-dimensional space using a kernel function. The final decision boundary is determined by the support vectors but can be optimized as needed. SVM can be used both for linearly and nonlinearly separable datasets with the use of polynomial or radial basis function respectively [28].

The following formula represents the decision function of a linear SVM:

$$f(x) = \text{sign}(w \cdot x + b) \tag{1}$$

2.2.2 Random forest classifier

The Random Forest Classifier is an ensemble classification algorithm composed of multiple decision trees used for both classification and regression tasks. RFC constructs multiple decision trees during the training phase. Each tree is built using a random subset of training data and features which helps in reducing overfitting. RFC aggregates the prediction of all trees to provide a final prediction [29]. RFC is robust

Table 1 Overview of vulnerability detection methods in Solidity smart contracts

Paper	Year	Method	Description
[12]	2024	Machine Learning model for opcode pattern analysis	Trained SVM-based model with real smart contracts dataset to analyze vulnerable hexcode patterns in contracts for etherlock
[34]	2023	Machine Learning & Fuzz Testing	Combines ML techniques with fuzz testing to detect smart contract vulnerabilities.
[35]	2023	Artificial Intelligence	Uses AI-driven analysis to enhance the detection of vulnerabilities in Solidity smart contracts.
[36]	2023	Multi-Layer Perceptron (MLP)	Implements MLP to detect vulnerabilities in real-time Solidity smart contracts.
[37]	2019	Machine Learning	Trained ML models on smart contract datasets to predict vulnerability occurrences.
[38]	2023	Graph Neural Networks (GNN)	Uses GNNs to model smart contract structures and identify vulnerabilities.
[39]	2022	Machine Learning	Employs a binary classification model trained on vulnerability datasets.
[40]	2023	Deep Learning (CNN, LSTM, CodeBERT)	Uses hybrid deep learning models to detect vulnerabilities without predefined rules.
[41]	2023	Deep Learning & Multimodal Fusion	Combines source code, opcodes, and control-flow graph data for vulnerability detection.
[42]	2023	Convolutional Neural Networks (CNN)	Detects reentrancy vulnerabilities using CNNs and compares with LSTM & GNN models.
[43]	2018	Opcode Pattern Analysis	Analyzes opcode sequences to detect vulnerabilities in smart contracts.
[44]	2022	Opcode Sequence Pattern Matching	Uses opcode sequence learning for unknown vulnerability detection.

Table 2 Contributions and limitations of selected smart contract vulnerability detection papers

Paper	Contribution	Limitations
[12]	Provides a high performing etherlock attack detection machine learning model integrated into a tool, providing instantaneous vulnerability detection	Limited dataset size and vulnerability detection limited to only etherlock vulnerability
[34]	Integrates ML & fuzz testing for better detection accuracy.	Limited dataset size may affect generalization.
[35]	AI-driven framework for Solidity security improvement.	Relies heavily on labeled datasets for training.
[36]	Applies MLP to real-time smart contract security monitoring.	Potential false positives due to overfitting in MLP models.
[37]	One of the first ML-based studies on smart contract security.	Lacks real-world deployment validation.
[38]	Uses GNNs to map complex smart contract structures.	Requires significant computation power for GNN processing.
[39]	Applies ML classification to detect smart contract vulnerabilities.	Struggles with imbalanced datasets.
[40]	Combines CNN, LSTM, and CodeBERT for high accuracy.	Higher computational cost compared to traditional methods.
[41]	Fuses multiple sources of data for enhanced detection.	Complex model training process requiring large datasets.
[42]	Uses CNN for reentrancy vulnerability detection.	Focuses only on reentrancy, lacks coverage for other vulnerabilities.
[43]	Opcode-based vulnerability detection method.	Limited scalability for new opcode-based exploits.
[44]	Uses opcode sequence learning for unknown vulnerabilities.	May fail to detect vulnerabilities beyond opcode patterns.

and can provide improved results with an increase of the trees and their depth but also depending on the dataset.

The following formula can be used to represent Random Forest Classifier:

$$D = \{(x_i, y_i)\}_{i=1}^{n} \tag{2}$$

The following formula represents the Gini index, representing the quality of the split. P(i) denotes ratio of entities in the dataset.

$$GI(N) = \sum_{i \neq j} P(\omega_i)P(\omega_i) \tag{3}$$

2.2.3 Decision tree

A Decision Tree is a type of supervised machine learning method that doesn't rely on fixed parameters and predicts outcomes for input data by making decisions at various nodes. Starting from the root node, it assesses specific features or attributes of the input and branches the data accordingly. This process iterates until it reaches a leaf node, which holds the final prediction. The choice of attributes for branching is determined by concepts like entropy and information gain, which quantify uncertainty and the change in uncertainty after a split. A higher information gain signifies a more beneficial split, and both entropy and information gain can be expressed mathematically [30].

The following formula represents Entropy where E(S) is entropy:

$$E(S) = \sum_{x \in N} p(x) \log_2 p(x) \tag{4}$$

The following formula represents information gain where p(x) is probability of x

$$IG(S, A) = E(S) - \sum_{i=0}^{n} p(x_i) \times E(S, N) \tag{5}$$

2.2.4 K-nearest neighbors

K-Nearest Neighbors (KNN) generates predictions based on K-nearest data points in a given dataset. The prediction for a given data point is based on the majority class of its K nearest neighbors in the feature space. The relationship is captured by calculating the Euclidean distance between the points. While it is simple, it is also computationally intensive. Additionally, the results are heavily affected by noise in the data and irrelevant features, more so than other algorithms [31].

The following formula is the Euclidean distance between two data points x and y in a multidimensional space:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2} \tag{6}$$

The following formula represents the predicted class label for a new data point by majority vote among its K nearest neighbors:

$$\hat{y} = argmax_c \sum_{i=1}^{K} I(y_i = c) \tag{7}$$

2.2.5 Long-short term memory

Long Short-Term Memory (LSTM) is a type of recurrent neural network (RNN) designed to handle sequential data by selectively retaining or discarding information over long sequences. Unlike traditional RNNs, LSTM uses gates (input, forget, and output) to control the flow of information [32], making it particularly effective for analyzing smart contract opcodes, where dependencies between instructions are critical for detecting vulnerabilities. By maintaining a cell state that updates dynamically, LSTM can capture both short-term patterns and long-range dependencies, improving the detection of smart contract vulnerabilities.

2.2.6 Convolutional neural networks

A Convolutional Neural Network (CNN) is a deep learning algorithm renowned for its efficacy in hierarchical feature extraction from structured input data. Primarily recognized for its applications in computer vision, CNNs systematically process data through specialized layers. Convolutional layers employ learnable filters (kernels) that scan the input, automatically identifying and extracting salient local features. Subsequently, pooling layers reduce the spatial dimensions of these feature maps, contributing to computational efficiency and enhancing the model's invariance to minor data shifts [33]. The high-level, abstract features derived from these operations are then passed to fully connected layers, which perform classification or regression based on the intricate patterns learned throughout the network.

3 Literature review

The detection of vulnerabilities in Solidity-based smart contracts has been a growing area of research, with numerous approaches being proposed to enhance security analysis. Approaches most closely inline with this proposed work can be broadly categorized into machine learning and artificial intelligence-based techniques and opcode and pattern analysis-based techniques. The former leverages computational intelligence to predict and classify vulnerabilities [37, 38, 40], whereas the latter focuses on static and dynamic code analysis through opcode interpretation [43, 44]. A thorough examination of these methods reveals strengths, limitations, and emerging trends. Table 1 summarizes the most recent machine learning approaches for smart contract vulnerability detection.

3.1 Machine learning & artificial intelligence-based approaches

Machine learning (ML) and artificial intelligence (AI) techniques have become increasingly popular for automating the detection of security vulnerabilities in Solidity smart contracts. Traditional security tools rely on rule-based static analysis, which often suffers from high false positive and false negative rates due to rigid detection heuristics [34]. To address these limitations, ML-based methods offer a data-driven approach that can adapt to evolving attack patterns.

Several works have explored ML for vulnerability detection, each employing different architectures and training methodologies. The proposed work by Tang et al. [40] introduces a hybrid CNN-LSTM-CodeBERT framework to automatically detect vulnerabilities without relying on predefined rules. This approach significantly enhances semantic

understanding by using CodeBERT, a model pre-trained on source code, to capture vulnerability characteristics more effectively. Similarly, Deng et al. [41] integrates multiple input sources, such as source code, operation code, and control-flow graphs (CFGs), to create a more robust detection model. This multimodal learning strategy improves accuracy by combining different representations of contract behavior. Table 2 below highlights contributions and inherent limitations of the proposed schemes in the literature.

The work proposed by Momeni et al. [37] provides an earlier perspective on ML-based detection, utilizing static code analyzers (Slither [45] and Mythril [46]), to generate training labels for classification models. While effective, this approach relies on predefined vulnerability detection tools, which may not generalize well to emerging attack vectors. Similiarly, Almobaideen et al. [12] provides an Etherlock vulnerability detection model using a labeled dataset of real contracts, labeled through vulnerability detection tools. In contrast, Haw Coli et al. [36] applies multi-layer perceptron (MLP) models to real-time Solidity contracts, offering a practical implementation with streaming vulnerability analysis.

Another notable technique is graph-based learning, as seen by Zhen et al. [38]. This method models contract dependencies and function interactions as graphs, allowing GNNs to infer complex relationships that may indicate security risks. The strength of this approach lies in its ability to capture inter-function dependencies, which is particularly useful for detecting reentrancy attacks and logical inconsistencies.

A key observation across ML-based approaches is the trade-off between accuracy and interpretability. While deep learning models such as CNNs, LSTMs, and transformer-based models (e.g., CodeBERT) exhibit superior accuracy in detecting vulnerabilities, they often lack explainability, making it challenging for developers to trust and verify their decisions as presented in the work by Bektas [42]. Furthermore, these models are data-dependent, meaning their effectiveness is limited by the availability of high-quality, labeled datasets. Research proposed by Gu et al. [39] attempts to address this by binary classification of vulnerabilities, but still faces the natural challenges associated with machine learning approaches such as handling newer vulnerabilities.

3.2 Opcode & pattern analysis-based approaches

Opcode and pattern analysis-based techniques take a lower-level approach, focusing on analyzing compiled bytecode instructions to detect vulnerabilities. This method is particularly useful in cases where source code is unavailable, as it allows researchers to inspect deployed smart contracts directly.

A foundational study in this domain is [43], which uses opcode pattern analysis to detect security risks at the Ethereum Virtual Machine (EVM) level. The authors apply symbolic execution and static analysis techniques to identify vulnerabilities in bytecode without requiring source code access. This approach provides high explainability, as detected vulnerabilities can be traced to specific opcode sequences.

Building upon opcode analysis, [44] introduces a sequence-learning model to identify potential threats based on opcode execution patterns. This method is particularly effective for recognizing new or unseen vulnerabilities, as it does not rely on predefined heuristics. However, one limitation of opcode sequence-based detection is its sensitivity to contract obfuscation-attackers can modify opcode structures to evade detection, reducing the model's efficacy.

One of the major advantages of opcode analysis methods is their robustness to source code modifications. Unlike ML-based techniques that depend on labeled training datasets, opcode-based approaches can analyze compiled smart contracts directly, making them more adaptable to real-world contract security evaluations. However, these techniques often suffer from scalability issues, as symbolic execution and static analysis can be computationally expensive. Additionally, they may fail to detect vulnerabilities that arise from complex contract interactions, which ML models can better capture.

Comparing the two approaches, ML-based techniques exhibit higher detection accuracy and can learn from real-world attack data, making them well-suited for identifying

Table 3 Common open source Solidity smart contract auditing tools

Tool	Description
Slither [45, 47]	Provides static analysis to identify reentrancy issues, uninitialized variables, and incorrect visibility settings.
Mythril [46]	Identifies vulnerabilities such as reentrancy, integer overflows, and unchecked calls using symbolic analysis.
Oyente [48, 49]	Detects transaction-order dependence, timestamp dependence, reentrancy vulnerabilities, and integer overflows using symbolic execution.
Securify [50, 51]	Uses formal verification to find issues like unhandled exceptions, reentrancy, and transaction-order dependence.
Maian [52, 53]	Detects suicidal, prodigal, and greedy contracts that can be killed, leak Ether, or lock Ether indefinitely.
Contract Fuzzer [54]	Uses fuzz testing to uncover vulnerabilities like reentrancy, gasless send, and timestamp dependence.
Conkas [55, 56]	Analyzes for concurrency-related vulnerabilities, such as reentrancy and race conditions, in Ethereum smart contracts.
Confuzzius [57, 58]	Uses concolic testing to detect vulnerabilities like arithmetic errors, reentrancy, and unauthorized access.

emerging vulnerabilities [40, 42]. However, they require substantial computational resources, are prone to false positives, and often suffer from limited interpretability [39]. Conversely, opcode-based methods offer explainable and rule-based detection, but they struggle with scalability and may fail to generalize to new attack patterns [43].

Several hybrid approaches have emerged that aim to combine the strengths of both methods. For instance, [41] integrates deep learning with opcode and CFG analysis, achieving higher robustness and precision. Future research should explore hybrid models that leverage both ML-driven insights and opcode-level detection to enhance real-time security monitoring.

Additionally, one of the most pressing challenges in ML-based vulnerability detection is the availability of labeled datasets. Some studies rely on synthetically generated datasets (e.g., SolidiFI benchmark), which may not fully represent real-world attack conditions. Efforts to curate high-quality, real-world vulnerability datasets will significantly enhance the performance and reliability of ML-based methods [37].

This paper presents a machine learning-based methodology for detecting smart contract vulnerabilities. It introduces a process for creating a labeled dataset using outputs from well-known auditing tools, and proposes a stacked ensemble model capable of detecting four common smart contract vulnerabilities which is novel to the literature. The results demonstrate the model's ability and potential to identify vulnerable opcode sequences with promising accuracy.

4 Dataset creation and preprocessing

The first step in creating the dataset is the collection of published smart contracts from the blockchain to create the labeled dataset. The Etherscan API was used for bulk retrieval of smart contracts from the Ethereum blockchain. The use of real smart contracts from the blockchain is crucial in order to develop a model that is based on vulnerabilities present on the actual blockchain, and not in a simulated environment. A model trained on simple contracts with intentional vulnerabilities written in the same manner, may not fare well in real testing, since the contracts and their semantics may differ from real implementation contracts.

Table 4 Example of bytecode, opcode and stripped hexcode conversion

Bytecode	Opcode	Stripped Hexcode
6080	PUSH1 80	60
61ABCD	PUSH2 0xABCD	61
01	ADD	01
56	JUMP	56

Through simple scripting, we may exclude contracts of specific compiler versions. Languages such as Solidity, Vyper, and Clarity explicitly state the compiler version as part of the syntax, however, this is not the case for some other languages such as TEAL.

Labeling of contracts consists of performing an automated vulnerability assessment on each contract and labeling it as vulnerable or non-vulnerable. In the event where several vulnerabilities must be detected, a multi-label classification can be considered, such that a vector like 1,0,1,0 is assigned to each contract. In this representation, each position in the vector corresponds to a specific type of vulnerability, with 1 indicating the presence of that vulnerability and 0 indicating its absence. This allows a contract to be labeled with multiple vulnerabilities simultaneously, rather than being restricted to a single binary classification. For example, let's consider the detection of three different vulnerabilities, vuln-1, vuln-2 and vuln-3. If a contract is vulnerable to all three, then it can be classified as {1,1,1}. If the contract is vulnerable to only vuln-1, then the corresponding output is {1,0,0} and so on. However, the main disadvantages of creating a multi-labeled dataset and machine learning model are the following:

Data-Related Issues

- Class Imbalance: Some labels appear more frequently than others, leading to biased predictions.
- Label Dependencies: Labels are often correlated, but many models treat them independently.
- Label Cardinality and Density: Variation in the number of labels per instance makes generalization difficult.***Algorithmic and Training Issues***

- High Computational Complexity: Multi-label classification requires handling multiple outputs, increasing training and inference time.
- Evaluation Metric Complexity: Traditional metrics are ineffective; specialized metrics like Hamming loss and Jaccard index are required.
- Threshold Selection: Choosing an appropriate probability threshold for label assignment is challenging and affects accuracy.

When labeling a smart contract as vulnerable or non-vulnerable, it is essential to use multiple smart contract auditing tools rather than relying on just one. Numerous open-source tools are available for auditing Solidity smart contracts, each with unique strengths and limitations. These tools may detect different types of vulnerabilities or overlap in their findings. If all tools used unanimously agree that a contract

is vulnerable (e.g., three out of three tools detect the vulnerability), it is reasonable to conclude with high confidence that the contract is indeed vulnerable. It is reasonable to also determine a contract to be vulnerable based on a majority consensus (e.g., two out of three tools detect the vulnerability). On the other hand, when considering the detection accuracy of the tools, using majority voting is not adequate.

It is critical to take into account the confidence level and accuracy of the tools involved. Confidence refers to how sure a tool is in its detection of a vulnerability. Some tools may produce results with a high success rate, meaning they rarely report false positives. Others may be less reliable, either due to limitations in detecting more complex contract behaviors or due to high sensitivity that leads to over-reporting. If two low-confidence tools detect a vulnerability, it would be more reasonable to defer to the result of a third high-confidence tool that did not detect the vulnerability. However, making decisions on a case-by-case basis for every contract is infeasible considering the size of the dataset. The common metric of most static analyzers is the false-positive rate. We can yield the True Positive Rate instead by evaluating $1 - FPR$. Using TPR, the following weighted decision-making methodology can be adopted. For each tool *i*, the weight *W* could be calculated as: $W_i = \frac{1}{\mathrm{TPR}_i + \epsilon}$ where TPR_i is the true positive rate for the tool *i*. ϵ is a small constant to prevent division by zero. After calculating the individual weights, we normalize them using the formula $\text{Normalized } W_i = \frac{W_i}{\sum_{j=1}^{n} W_j}$. Finally, to determine the overall vulnerability score, we compute $\text{Vulnerability Score} = \sum_{i=1}^{n} \quad (\text{Normalized } W_i \times T_i) \quad$, where T_i is an indicator variable of whether tool *i* reports a vulnerability. This approach takes into account the confidence rate of each tool, providing weighted vulnerability labeling. In a scenario where a common relevant evaluation metric is not shared by any set of tools, voting consensus should be adopted. Table 3 below describes open source static analyzers commonly used for smart contract auditing.

4.1 Dataset preprocessing

The next step is the creation of an input dataset from the collected smart contracts. The difficulty of using the source code as a data instance in itself stems from the source code being a string, which is difficult to process for several reasons. First, smart contracts as strings can have high dimensionality and complexity, with numerous keywords, comments, and variable names that may not contribute to identifying vulnerabilities. This leads to the curse of dimensionality, making the model harder to train and more computationally expensive. Second, a raw string lacks semantics, meaning the model cannot easily detect the underlying patterns or logic that may cause vulnerabilities, such as control flow or data flow issues.

Third, the raw string representation lacks proper tokenization and preprocessing, so irrelevant parts like comments or whitespace can add noise to the data and make classification harder. Fourth, different developers use inconsistent naming conventions for variables and functions, and a raw string input will present a wide variety of irrelevant names, making it difficult for the model to generalize. Finally, the string contains irrelevant tokens such as comments and whitespace, which do not contribute to determining vulnerability, adding unnecessary noise and reducing the model's efficiency. As such, the use of the source directly is not suitable. We employ the use of converting operational code instructions to hexadecimal representations of the code for processing.

4.1.1 Converting source code to hexcode

To create a labeled smart contract dataset, we employ a process that converts the source code into a format suitable for machine learning models. Firstly, after collecting the source code of each smart contract, we compile the Solidity code to generate the corresponding bytecode. This bytecode represents the low-level instructions executed by the Ethereum Virtual Machine (EVM). The EVM uses over a hundred distinct operation codes (opcodes), which cover a variety of operations including arithmetic, bitwise, stack manipulation and cryptographic operations. A complete list of EVM opcodes can be found here [59]. Each bytecode instruction is directly mapped to a specific opcode, and the EVM interprets and executes these opcodes sequentially. In a given stream of bytecode, the first byte is checked. The byte is internally mapped to an opcode. If it is an opcode that expects an argument, then the second 1 byte upto 32 bytes (depending on opcode) is an argument of the first byte. Otherwise, if it is an opcode that does not expect an argument, then the second bytecode after the first is simply another opcode.

Secondly, We further transform the bytecode to an input with reduced complexity and focus on control/data flow patterns by retaining only the opcode mnemonics (e.g., PUSH1, SLOAD, CALL) while discarding operand values. This abstraction prevents the model from overfitting to contract-specific constants and enhances generalization across diverse smart contracts. Since operand values introduce high entropy and sparsity without meaningful semantic contribution, stripping them improves the signal-to-noise ratio of the input sequence. This methodology is standard and necessary when creating input based on opcodes for

analysis [60–63]. Table 4 demonstrates the transformation of bytecode to opcode, to stripped hexcode.

For example, in an operation such as PUSH1 0x80, we are only concerned with the PUSH1 opcode, not its associated value. By stripping these values, we create a more refined and standardized dataset that focuses on the structural patterns of the contract's execution logic. An original bytecode representation like 6080604052348015610010576000 80FD5B5061014E806100206000396000F3FE will be stripped into the following vectorized hexcode representation: 60 60 52 34 80 15 61 57 60 80 FD 5B 50 61 80 61 60 39 60 F3 FE. The methodology remains similar whether dealing directly with opcode instructions or the raw bytecode. If handling opcode instructions (e.g., from a disassembled view), the opcode and its value can be easily separated, allowing the first element (the opcode) to be collected while ignoring the value. If directly processing a bytecode stream, the process involves identifying opcodes and skipping their respective argument bytes. In essence, regardless of whether we are dealing with the source code of the contract, direct bytecode, or opcode instructions, the core stripping process is consistent. The pseudocode presented in Listing 7 demonstrates the methodology for converting opcode instructions to stripped hexcode. Similarly, the pseudocode presented in Listing 8 demonstrates the methodology for directly processing a continuous bytecode string to obtain stripped hexcode. Both pseudocodes correctly account for PUSH1 to PUSH32 opcodes, which contain arguments ranging from 1 to 32 bytes in size.

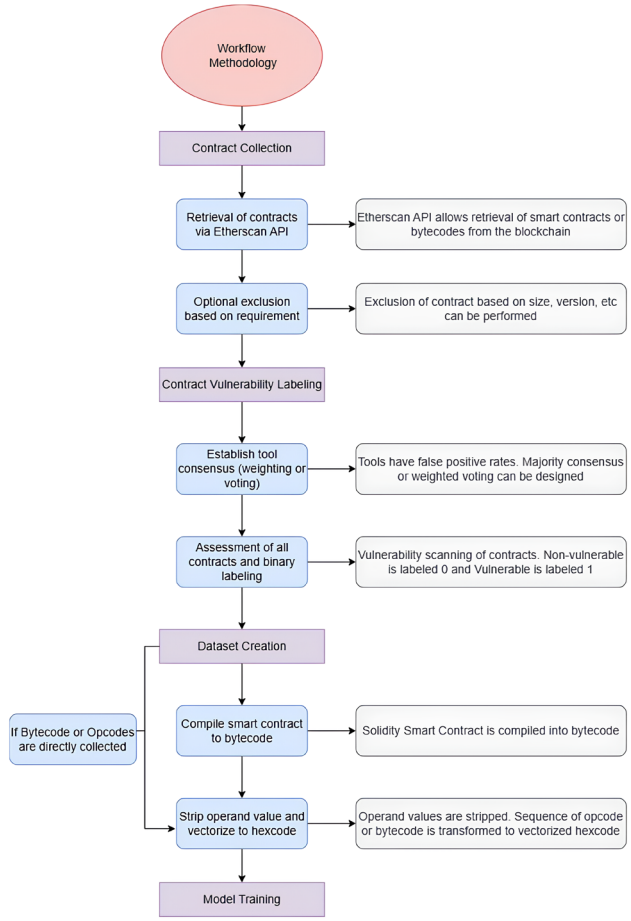


Fig. 1 Workflow methodology of the proposed approach, illustrating the steps from contract collection to training the model

```
Input: WORDLIST (a list of EVM instruction strings, e.g., ["PUSH1 0x80", "ADD", "
    CALLVALUE"])
Output: STRIPPED_BYTECODE_HEX_REPRESENTATION (a continuous string of hexadecimal
    characters representing only the opcodes, e.g., "600134")

// Pre-requisite: You need a mapping from opcode mnemonics to their hex values.
// Example: OPCODE_MNEMONIC_TO_HEX_MAP = {"STOP": "00", "ADD": "01", "PUSH1": "60",
    ...}

1. Initialize an empty list called CLEANED_LIST_OF_MNEMONICS.
2. Initialize an empty string called STRIPPED_BYTECODE_HEX_REPRESENTATION.

// --- Step A: Extract only opcode mnemonics from instruction strings ---
3. For each INSTRUCTION_STRING in WORDLIST:
    a. Split INSTRUCTION_STRING by whitespace into a list called PARTS.
       (e.g., if INSTRUCTION_STRING is "PUSH1 0x80", PARTS becomes ["PUSH1", "0x80"
           ])
    b. If PARTS is not empty:
        i. Get the OPCODE_MNEMONIC, which is the first element of PARTS (PARTS[0]).
        ii. Append OPCODE_MNEMONIC to CLEANED_LIST_OF_MNEMONICS.
           (This effectively discards any subsequent elements in PARTS, which would
                be the value/operand)

// --- Step B: Convert stripped mnemonics back to hexadecimal bytecode ---
4. For each MNEMONIC in CLEANED_LIST_OF_MNEMONICS:
    a. Look up the hexadecimal representation of the MNEMONIC:
       Set OPCODE_HEX_VALUE to OPCODE_MNEMONIC_TO_HEX_MAP[MNEMONIC].
       (e.g., if MNEMONIC is "PUSH1", OPCODE_HEX_VALUE becomes "60")
    b. Append OPCODE_HEX_VALUE to STRIPPED_BYTECODE_HEX_REPRESENTATION.

5. Return STRIPPED_BYTECODE_HEX_REPRESENTATION.
```

```
Input: RAW_BYTECODE_STRING (a continuous string of hexadecimal characters, e.g., "
    6080604052...")
Output: STRIPPED_OPCODE_HEX_STRING (a continuous string of hexadecimal characters,
    representing only the opcodes, e.g., "606052...")

// Helper function: Converts a 2-character hex string to its integer value.
// Example: HEX_TO_INT("60") returns 96
// This function needs to be implemented separately based on your programming
    language.

1. Initialize an empty string called STRIPPED_OPCODE_HEX_STRING.
2. Initialize an integer called CURRENT_POSITION_IN_BYTECODE to 0.

3. While CURRENT_POSITION_IN_BYTECODE < length of RAW_BYTECODE_STRING:
    a. Extract the current opcode hex representation:
       Set CURRENT_OPCODE_HEX to the substring of RAW_BYTECODE_STRING starting
       at CURRENT_POSITION_IN_BYTECODE and extending for 2 characters.
       (This gets one byte, e.g., "60", "52", "61")

    b. Convert the current opcode hex to an integer:
       Set CURRENT_OPCODE_INT to HEX_TO_INT(CURRENT_OPCODE_HEX).

    c. Append the CURRENT_OPCODE_HEX to STRIPPED_OPCODE_HEX_STRING.
       (We keep the opcode own hex representation.)

    d. Determine how many hexadecimal characters to advance the
        CURRENT_POSITION_IN_BYTECODE:
       i. If CURRENT_OPCODE_INT is between 0x60 (decimal 96, PUSH1) and 0x7F (
           decimal 127, PUSH32) (inclusive):
          // This opcode is a PUSH instruction and takes an immediate value.
          Set NUM_BYTES_TO_PUSH to (CURRENT_OPCODE_INT - 0x60) + 1.
          // Advance past the opcode itself (2 hex chars) AND its value (
              NUM_BYTES_TO_PUSH * 2 hex chars).
          Increment CURRENT_POSITION_IN_BYTECODE by (2 + (NUM_BYTES_TO_PUSH * 2)).
       ii. Else (for all other EVM opcodes that do not take immediate arguments
           from the bytecode stream):
          // This opcode does not have an immediate value following it.
          // Advance only past the opcode itself (2 hex chars).
          Increment CURRENT_POSITION_IN_BYTECODE by 2.

4. Return STRIPPED_OPCODE_HEX_STRING.
```

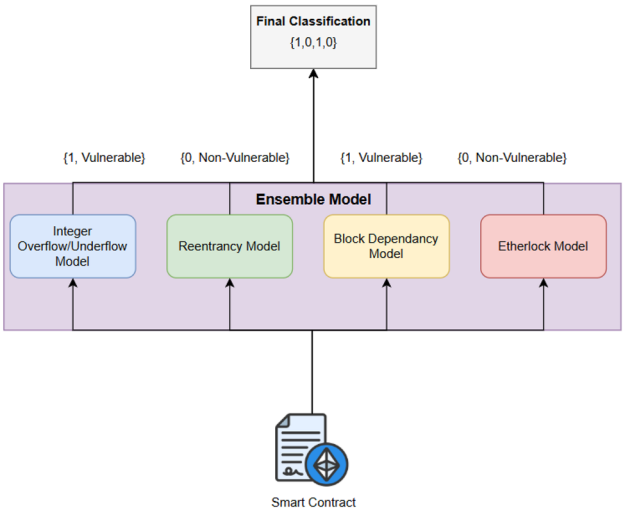


Fig. 2 Stacked ensemble model for smart contract vulnerability detection. Individual classifiers detect specific vulnerability types, and their outputs are combined to produce a final multi-label classification

Table 5 Summary of vulnerability datasets

Dataset	Total	Non-vulnerable	Vulnerable	% Vulnerable
Etherlock	6269	6000	269	4.29%
Block dependency	6114	6000	114	1.59%
Integer (Under/Overflow)	6937	6000	937	15.62%
Reentrancy	6293	6000	293	0.47%

Table 6 Training time comparison across models and datasets (in minutes)

Dataset	Model	Time per fold	Total training time
EtherLock	KNN	3	18
	DecisionTree	4.5	27
	SVM	22.5	135
	RandomForest	7.5	45
	XGBoost	6	36
	LSTM	7.5	45
	CNN	9	54
BlockDependency	KNN	6	36
	DecisionTree	9	54
	SVM	45	270
	RandomForest	15	90
	XGBoost	12	72
	LSTM	15	90
	CNN	18	108
IntegerOverflow	KNN	9	54
	DecisionTree	13.5	81
	SVM	67.5	405
	RandomForest	22.5	135
	XGBoost	18	108
	LSTM	22.5	135
	CNN	27	162
Reentrancy	KNN	4.5	27
	DecisionTree	6.75	40.5
	SVM	33.75	202.5
	RandomForest	11.25	67.5
	XGBoost	9	54
	LSTM	11.25	67.5
	CNN	13.5	81

The general steps are summarized below:

- Retrieval of smart contracts from the blockchain using Etherscan API.
- Label source code for vulnerabilities using Solidity smart contract auditing tools.
- Compile the source into bytecode.
- Strip the bytecode operand values and convert to vectorized hexcode.
- Compile the final bytecode and label as vulnerable or non-vulnerable

This approach is efficient and preserves all essential information from the original source code that could indicate potential vulnerabilities. By transforming high-level source code into a processable form based on its execution logic, we ensure that no crucial information is lost. The sequence and patterns in the opcodes are directly linked to the contract's functionality and potential vulnerabilities, making them ideal for use in machine learning models. Unlike other approaches in the literature that may rely on higher-level or contextual data, our methodology focuses on the intrinsic logic of the smart contract, ensuring a clear, standardized representation for model training in addition to being faster. Moreover, this method satisfies maintaining privacy of contracts by working on a bytecode level that does not reveal the source code post data-processing.

Table 7 Hypertuning parameter values of the algorithms, optimized via Grid Search

Algorithm	Parameter	Value
SVM	gamma	scale
	kernel	rbf
	c	10.0
	random_state	42
	probability	True
DT	random_state	42
	max_depth	None
	min_samples_split	2
	min_samples_leaf	1
	max_features	None
RFC	n_estimators	100
	max_depth	42
KNN	n_neighbors	5
LSTM	hidden_units	256
	num_layers	2
	dropout	0.3
	batch_size	64
	learning_rate	0.0005
	sequence_length	100
	bidirectional	True
	optimizer	Adam
	loss_function	Binary Cross-Entropy
	activation	Sigmoid
CNN	vocab_size	len(tokenizer.word_index) + 1
	embed_dim	32
	num_filters	64
	kernel_size	5
	dropout	0.5
	fc_output	1
	activation	Sigmoid
	optimizer	Adam
	learning_rate	0.001
	loss_function	BCELoss
	epochs	5 (main), 3 (cross-validation)
	batch_size	8

For when new opcode instructions are adopted, the translation of these opcodes can be added into the EVM mnemonic dictionary of the code, ensuring that the system remains up-to-date and extensible. This allows the model to seamlessly incorporate and learn from newly introduced instructions without requiring changes to the core processing pipeline. For example, newer opcodes introduced in versions Solidity 0.8.x+ Constantinople, Berlin and Shanghai

Table 8 Description of machine learning evaluation metrics used

Metric	Definition
Accuracy	The ratio of correctly predicted instances to the total number of instances.
Precision	The ratio of true positives to all predicted positive instances.
Recall	Also known as sensitivity; the ratio of true positives to all actual positive instances.
F1-score	The harmonic mean of precision and recall, balancing both metrics.
AUC-ROC	Area Under the ROC Curve; measures a model's ability to distinguish between classes.
TP	True Positives: instances correctly predicted as positive.
TN	True Negatives: instances correctly predicted as negative.
FP	False Positives: instances incorrectly predicted as positive.
FN	False Negatives: instances incorrectly predicted as negative.
P-interval	The range of plausible values for a population proportion based on sample data.
Confidence interval	A range of values that is likely to contain an unknown population parameter.

can be processed since the opcode translation is added to the EVM mnemonic dictionary in the code.

This methodology does not require to explicitly address proxy contracts; This is because it operates at the opcode level, identifying specific instruction patterns indicative of the listed vulnerabilities. Therefore, regardless of whether the analyzed bytecode belongs to the administrative logic of a proxy contract or a delegated implementation contract, our model applies its detection capabilities to the provided code segment.

In terms of inline assembly, this approach handles contracts with inline assembly correctly. This is because Solidity's inline assembly is compiled into standard EVM bytecode, just like Solidity itself. Once compiled, the source of the instruction (Solidity or inline assembly) is irrelevant as the resulting bytecode consists of the same opcodes. Since this approach operates at the opcode level, it fully captures both standard Solidity and inline assembly logic uniformly.

4.1.2 Opcode overfitting and generalization

A key challenge in training machine learning models for vulnerability detection is the risk of overfitting, where a model memorizes specific patterns in the training data rather than learning generalizable features. In the context of opcode-based analysis, this could manifest in two primary ways. First, a model could overfit to literal operand values, associating a vulnerability with a specific instruction like PUSH1 0x60 instead of the opcode PUSH1 itself. This risk is effectively mitigated by our preprocessing method, which strips these values and retains only the opcode mnemonics. Second, a model could overfit to specific opcode

Table 9 Macro average performance metrics of each model across all datasets. Best result across every dataset is highlighted

Vulnerability	Model	Accuracy	F1-Score	ROC AUC	Recall	Precision
lightgreen Block dependency	SVM	0.786	0.780	0.899	0.820	0.780
	RFC	0.893	0.880	0.966	0.910	0.870
	DT	0.853	0.850	0.861	0.860	0.820
	LSTM	0.671	0.630	0.676	0.630	0.620
	KNN	0.650	0.660	0.826	0.650	0.720
	CNN	0.825	0.810	0.915	0.840	0.800
lightgreen Etherlock	SVM	0.990	0.995	0.999	0.990	0.990
	RFC	0.997	0.999	0.999	0.995	0.995
	DT	0.984	0.980	0.984	0.980	0.980
	LSTM	0.897	0.890	0.949	0.880	0.870
	KNN	0.890	0.820	0.902	0.830	0.810
	CNN	0.984	0.980	0.996	0.980	0.980
lightgreen Integer Underflow/Overflow	SVM	0.821	0.820	0.925	0.840	0.810
	RFC	0.933	0.930	0.983	0.940	0.920
	DT	0.894	0.890	0.904	0.900	0.870
	LSTM	0.796	0.780	0.878	0.830	0.790
	KNN	0.830	0.760	0.789	0.720	0.870
	CNN	0.810	0.801	0.938	0.860	0.800
lightgreen Reentrancy	SVM	0.732	0.730	0.860	0.790	0.750
	RFC	0.821	0.820	0.925	0.840	0.810
	DT	0.790	0.780	0.794	0.790	0.770
	LSTM	0.395	0.390	0.513	0.520	0.540
	KNN	0.741	0.710	0.773	0.720	0.700
	CNN	0.662	0.660	0.895	0.750	0.720

Table 10 F1-score performance summary of models across vulnerabilities, including standard deviation and 95% confidence intervals

Vulnerability	Model	6-FCV Mean F1	Std. Dev.	CI Lower	CI Upper	CI Width
Block Dependancy	CNN	0.7910	0.0199	0.7751	0.8069	0.0318
	DT	0.8505	0.0654	0.7982	0.9029	0.1047
	KNN	0.7989	0.0233	0.7803	0.8176	0.0373
	LSTM	0.0456	0.0338	0.0186	0.0726	0.0540
	RFC	0.8958	0.0545	0.8522	0.9394	0.0872
	SVM	0.8072	0.0368	0.7778	0.8366	0.0588
Etherlock	CNN	0.7924	0.0543	0.7489	0.8358	0.0869
	DT	0.9863	0.0060	0.9815	0.9911	0.0096
	KNN	0.8868	0.0522	0.8450	0.9286	0.0836
	LSTM	0.7290	0.0629	0.6787	0.7793	0.1006
	RFC	0.9960	0.0043	0.9925	0.9994	0.0069
	SVM	0.9815	0.0031	0.9790	0.9840	0.0050
Integer	CNN	0.9236	0.0047	0.9198	0.9274	0.0076
	DT	0.8913	0.0348	0.8635	0.9192	0.0557
	KNN	0.5892	0.0233	0.5706	0.6079	0.0373
	LSTM	0.8379	0.0089	0.8308	0.8451	0.0143
	RFC	0.9441	0.0407	0.9115	0.9767	0.0652
	SVM	0.8279	0.0096	0.8202	0.8356	0.0154
Reentrancy	CNN	0.8018	0.0213	0.7848	0.8189	0.0341
	DT	0.7953	0.0179	0.7810	0.8096	0.0286
	KNN	0.6947	0.0443	0.6593	0.7302	0.0709
	LSTM	0.4927	0.1142	0.4013	0.5841	0.1828
	RFC	0.8444	0.0147	0.8326	0.8563	0.0237
	SVM	0.7778	0.0113	0.7688	0.7869	0.0181

Table 11 Paired P-value Results: Comparison of top two performing models on each dataset

Vulnerability	Best model	Second best model	t-statistic	p-value
Block Dependancy	RFC	DT	3.735	0.0286
Etherlock	RFC	SVM	7.030	0.0033
Integer underflow/overflow	RFC	DT	1.812	0.1368
Reentrancy	RFC	DT	8.382	0.0005

Table 12 Average execution time per tool in milliseconds (lower is better)

Tool	**Execution time (ms)**
Honeybadger	98000
Maian	316000
Manticore	1468000
Mythril	84000
Osiris	34000
Oyente	30000
Securify	397000
Slither	5000
Smartcheck	10000
Our Model	**17.90**

sequences. For instance, it might learn to associate a pattern like PUSH1 JUMP DUP1 ADD with a vulnerability. An overfit model would fail to detect the same vulnerability if it were implemented with a slightly different but functionally equivalent sequence, such as PUSH1 SWAP1 DUP1 ADD, because it has memorized the literal sequence rather than understanding the underlying control flow. This second form of overfitting is mitigated through the use of a diverse and sparse dataset, which ensures the model is exposed to a wide variety of opcode patterns and prevents it from relying on a small number of memorized sequences. By combining a value-agnostic preprocessing method with a rich dataset, the model performance on a held-out test set demonstrates its generalization capabilities. Fig. 1 below demonstrates the workflow methodology of collecting Smart Contracts as source code, performing preprocessing, and generating machine learning processable input.

4.2 Dataset description

The final dataset contains two attributes:

1) Stripped Hexcode: Hexadecimal format of the value stripped operation codes of the Solidity smart contract.
2) Vulnerable: Binary classification of whether the corresponding hexcode is vulnerable, where 1 and 0 represent vulnerable and non-vulnerable, respectively.

Table 5 Below describes the dataset in terms of total contracts, number of non-vulnerable contracts, number of

Table 13 True and False Positive Rates of Static Analyzers in Comparison to Our Proposed Model

Tool	EL TP	EL FP	RE TP	RE FP	BD TP	BD FP	IO/U TP	IO/U FP
Conkas	-	-	-	23.2%	-	-	-	-
Honeybadger	-	-	0%	-	-	-	0%	-
Maian	69.0%	17.2%	0%	-	-	-	0%	-
Manticore	-	-	25%	-	-	-	22%	-
Mythril	0%	0%	62–90%	0%	42%	0%	68–94%	22%
Osiris	-	-	62%	-	-	-	50%	-
Oyente	0%	0%	62–72%	0%	0%	0%	55–63%	6.4–25%
Securify	-	-	62%	25.0%	-	-	0%	-
Solhint	-	-	-	91.3%	-	-	-	-
Slither	-	-	88%	11.0%	-	-	0%	-
Smartcheck	-	-	62%	73.6%	-	-	22%	-
Confuzzius	100%	0%	90%	0%	100%	0%	94%	0%
Contractfuzzer	100%	-	100%	-	96.3%	-	-	-
Our Model	**99.5%**	**0.08%**	**89%**	**20.0%**	**94.0%**	**12.7%**	**98.0%**	**8.5%**

Table 14 Comparison of performance metrics of our model against most similar approaches in the literature and multi-label approach

Model	Algorithm	Accuracy	Precision	Recall	F1
[62]	K-means Clustering	–	0.590	0.530	0.560
[60]	Random Forest Classifier	0.333	0.272	0.467	0.343
Our model	Random Forest Classifier	0.790	0.810	0.850	0.820

vulnerable contracts, and percentage of vulnerable contracts in each dataset.

In addition to our collection of smart contracts, the dataset was expanded by combining it with the smart bugs dataset [64, 65]. Overall, the dataset consists of 7,613 unique contracts. As seen from table, naturally, the ratio of vulnerable to non-vulnerable contracts is extremely low. Consequently, the dataset is heavily imbalanced. For training, oversampling was performed to balance the classes. SMOTE (Synthetic Minority Over-sampling Technique) was used to perform the oversampling, which, is a popular technique used to handle imbalanced datasets in classification tasks. Instead of simply duplicating minority class examples, SMOTE generates new synthetic samples by interpolating between existing ones. A train-test ratio of 80:20 was adopted for all models. The limited dataset size, primarily due to resource constraints, ensures that the runtime for all models remains feasible for our experimentation with limited computational resources. The training duration of all models across the datasets in provided in Table 6. The dataset and source code of the model are published on Kaggle https://www.kaggle.com/datasets/rha4365/vulnerable-solidity-smart-contracts-dataset/data.

5 Proposed model and results

In this section we describe the proposed models, hyper-tuning parameters, and in-depth discussion of the results including compilation of the model into a tool.

5.1 Ensemble model stacking

In order to maximize the performance of the tool, an ensemble model was developed. Ensemble learning is a powerful technique that improves predictive accuracy by combining multiple models. One of the most effective ensemble strategies is model stacking. Model stacking is a technique where several base models first make independent predictions, and a meta-model then learns to optimize the final decision. Alternatively, in the case of multi-label classification, the separate binary classification of each model is considered on its own. This serves the purpose of solving issues associated with multi-label models, such as discussed in Sect. 4. Table 6

In the context of smart contract vulnerability detection, it can be the case that a single smart contract suffers from several vulnerabilities. Therefore, the prediction of each model is true. The major advantage and motivation for using ensemble models is to reduce the generalization error of the prediction when multiple classes exist within a data set. Such a technique allows one to maintain the accuracy and performance of each separate model while maintaining the multi-label functionality [66]. Each separate model performs a binary classification task between vulnerable and non-vulnerable represented by the output of 1 and 0 respectively. Combining the output of each model produces the final label represented with 1's and 0's, each position indicating the result of the respective model. For example, the output of 1,0,1,0 means that the provided contract is vulnerable to Integer Overflow/Underflow, Block Dependency,

but not Reentrance and Etherlock. Fig. 2 below visualizes the the ensemble model stack, producing a single multi-label classification.

Several supervised machine learning algorithms were tested for each classification task: SVM, DT, LSTM KNN, RFC, CNN. This selection was made due to practical resource constraints that precluded an exhaustive evaluation of all possible classification algorithms. The chosen models possess strengths relevant to bytecode pattern recognition and have also been frequently highlighted in the literature for opcode analysis and similar tasks. Although powerful gradient boosting algorithms such as XGBoost are recognized for their performance, they are prone to overfitting. Table 7 describes the parameters used for each algorithm.

Model stacking is particularly powerful in this scenario due to its modular nature. When the detection of an additional vulnerability is required, a separate model can be trained on a newly created dataset and seamlessly integrated into the ensemble. In standard multi-label classification, incorporating a new dataset typically requires re-training and re-evaluating the model. Moreover, increasing the number of classification labels can negatively impact the model's overall accuracy due to added complexity and potential class imbalance.

Model parameters were optimized through the use of Grid Search. Grid Search is a hyperparameter tuning technique that systematically evaluates all possible combinations of predefined parameter values to identify the optimal configuration for a model.

5.2 Evaluation metrics

The model was evaluated using the standard metrics for machine learning models. Accuracy, Precision, Recall and F1-score were used as the main metrics to evaluate the general performance of the proposed model, represented in Eqs. 8–11. The confidence in the model is assessed using the confidence interval, calculated according to Eq. 12. Comparison of different models is assessed using the P-interval value defined in 14 which is calculated from the t-distribution with n-1 degrees of freedom, defned in Eq. 13.

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + TN + FP} \tag{8}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{9}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{10}$$

$$\text{F1} = 2 \times \frac{P \times R}{P + R} \tag{11}$$

$$\text{CI} = \bar{x} \pm z \cdot \frac{s}{\sqrt{n}} \tag{12}$$

$$\text{t} = \frac{\bar{d}}{s_d/\sqrt{n}} \tag{13}$$

$$\text{p} = 2 \cdot P(T_{n-1} > |t|) \tag{14}$$

These metrics provide a comprehensive evaluation of a model's performance on a classification task. Accuracy measures the overall correctness of the model's predictions, while precision indicates how many of the positively predicted cases were actually correct. Recall assesses the model's ability to detect all actual positive instances. The F1-score balances precision and recall, especially useful in imbalanced datasets. Confidence intervals (CI) quantify the uncertainty in the performance metrics, showing the range in which the true score likely lies. The t-statistic and p-value are used in statistical hypothesis testing to determine whether differences in performance between models are significant or could have occurred by chance. Together, these metrics help validate the effectiveness and reliability of the model in identifying the target classes within the dataset. Table 8 below defines all machine learning metrics used to assess the performance of the proposed model stack.

5.3 Model results, analysis and comparison

The following section provides all the results of the models tested on the four datasets. We also present a comparison in run-time with other static analyzer tools, false and true positive rates against static analyzers, as well as comparison of ML metrics with the most similar works in the literature. Table 9 provides the macro average test results of every model across all datasets. Table 10 provides the confidence intervals and 6-Fold Cross F1 mean validation score. Table 11 provides a paired P-value comparison of the top two models in each dataset. Comparison of runtime execution speed between our model and common static analyzers is presented in Table 12. Further comparison with static analyzers in terms of true positive and false positive rates is provided in Table 13. Finally, Table 14 compares our model with two most recent models in the literature adopting a closely similiar methodology.

The evaluation of our machine learning models, designed to identify specific smart contract vulnerabilities by learning patterns within stripped bytecode representations, provides critical insights into their effectiveness and generalizability. As detailed in Table 8, the F1-score represents the mean F1 of a 6-fold cross-validation, offering a robust measure of performance that balances precision and recall. Table 9

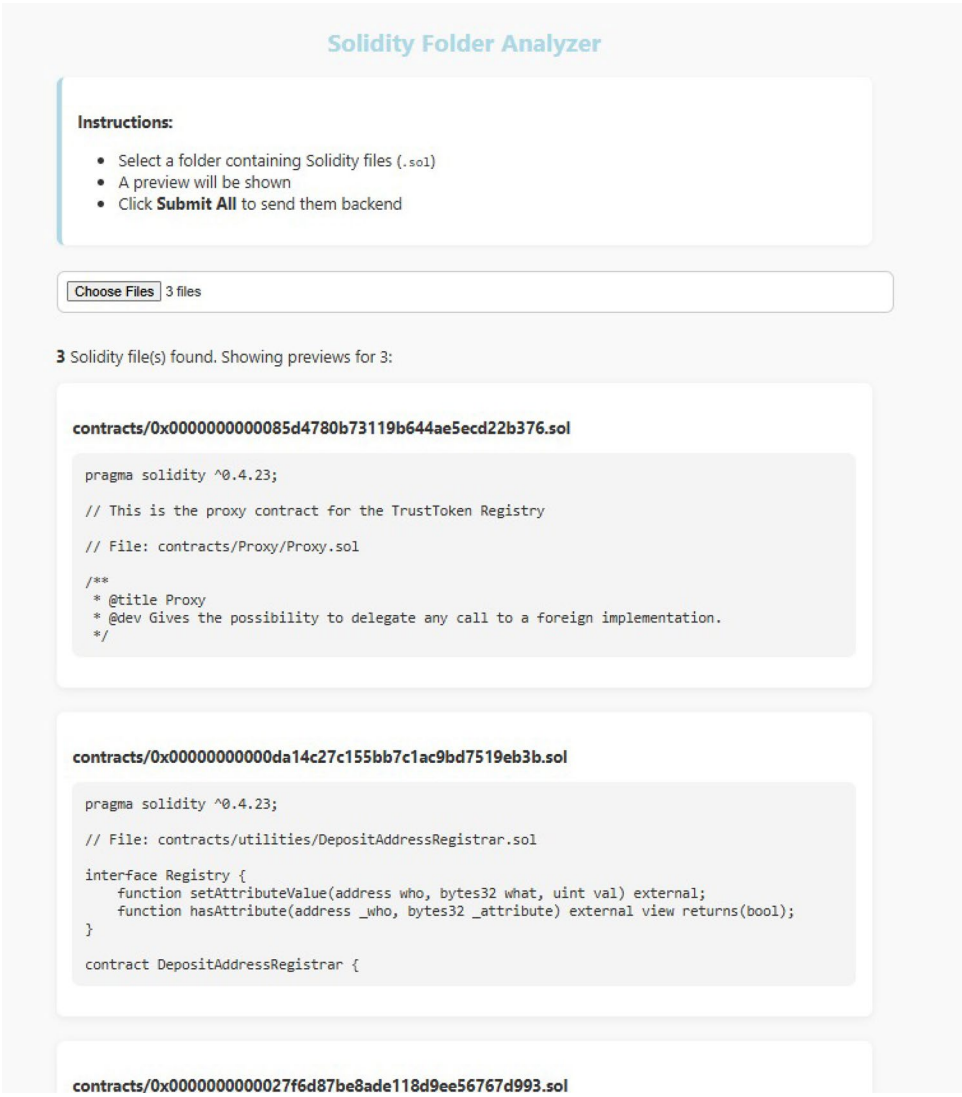
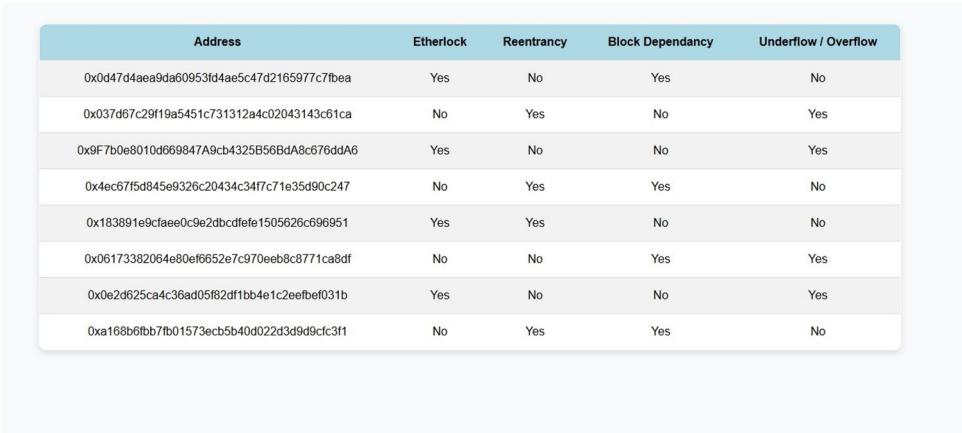


Fig. 3 User interface, uploading contracts for scanning



Address	Etherlock	Reentrancy	Block Dependancy	Underflow / Overflow
0x0d47d4aea9da60953fd4ae5c47d2165977c7fbea	Yes	No	Yes	No
0x037d67c29f19a5451c731312a4c02043143c61ca	No	Yes	No	Yes
0x9F7b0e8010d669847A9cb4325B56BdA8c676ddA6	Yes	No	No	Yes
0x4ec67f5d845e9326c20434c34f7c71e35d90c247	No	Yes	Yes	No
0x183891e9cfaee0c9e2dbcdfefe1505626c696951	Yes	Yes	No	No
0x06173382064e80ef6652e7c970eeb8c8771ca8df	No	No	Yes	Yes
0x0e2d625ca4c36ad05f82df1bb4e1c2eefbef031b	Yes	No	No	Yes
0xa168b6fbb7fb01573ecb5b40d022d3d9d9cfc3f1	No	Yes	Yes	No

Fig. 4 Example scan results of smart contracts through user interface

presents the macro average performance metrics across all datasets, highlighting the best results.

For Etherlock vulnerabilities, the models exhibited exceptional performance. RFC consistently achieved near-perfect scores, with an F1-Score of 0.999 and a mean F1 of 0.9960, coupled with an extremely low standard deviation of 0.0043 and a narrow 95% confidence interval (0.9925-0.9994). This stability and high performance suggest that etherlock patterns, often involving distinct opcodes related to SELFDESTRUCT or unhandled Ether, are remarkably consistent and uniquely identifiable within the stripped bytecode. SVM and DT models also delivered strong mean F1 scores (0.9815 and 0.9863 respectively) with very narrow confidence intervals, reinforcing the high detectability of this vulnerability type. Studies such as [67] and [58] have reported similar high detection rates (Table 13), consequently due to lack of opcode patterns related to releasing Ether.

Similarly, the detection of Integer Underflow/Overflow vulnerabilities showcased robust performance, with the RFC again leading. It achieved a mean F1 of 0.9441 with a standard deviation of 0.0407, and a confidence interval spanning 0.9115 to 0.9767. This strong showing indicates that the opcode sequences associated with arithmetic and stack manipulations characteristic of integer vulnerabilities form highly recognizable patterns. The Decision Tree model also performed commendably (mean F1: 0.8913, std dev: 0.0348), suggesting that these vulnerabilities can be characterized by clear decision boundaries in the opcode feature space. CNN also showed strong results here (mean F1: 0.9236) with very low variability (std dev: 0.0047).

In contrast, the models demonstrated more varied and generally lower performance for Block Dependency and particularly Reentrancy vulnerabilities. For Block Dependency, while RFC (mean F1: 0.8958) and DT (mean F1: 0.8505) still performed well, other models like LSTM (mean F1: 0.0456) and KNN (mean F1: 0.7989) showed considerably weaker results. The broad confidence interval for LSTM (0.0186-0.0726) highlights its unstable and poor performance, suggesting that block dependency patterns, which rely on specific block-related opcodes (TIMESTAMP, NUMBER, BLOCKHASH), might be more context-dependent or less uniquely defined by simple sequential opcode patterns.

Reentrancy proved to be the most challenging vulnerability, with all models struggling notably compared to other types. The LSTM model performed particularly poorly (mean F1: 0.4927), exhibiting a very wide confidence interval (0.4013-0.5841) and high standard deviation (0.1142), indicating significant instability and low reliability. Even the best performing RFC for reentrancy achieved a mean F1 of 0.8444, which, while respectable, is lower than its performance on other vulnerability types. This difficulty in detecting reentrancy from stripped bytecode alone is attributed to its complex nature, which involves intricate control flow, external calls, and dynamic state changes that are not fully captured by static opcode sequences without deeper data flow or inter-procedural analysis which is where static analyzers can boast better performance. The general trend of wider confidence intervals and higher standard deviations across models for Reentrancy (e.g., KNN's 0.6947 F1 with 0.0443 std dev and 0.6593-0.7302 CI) further underscores the inherent complexity and variability in detecting this vulnerability type from opcode patterns alone.

To further verify the statistical significance of the observed performance differences, paired t-tests were conducted comparing the top two performing models for each vulnerability dataset, with the results summarized in Table 11. This test indicates whether the superior performance of the best model is statistically significant.

For Etherlock vulnerabilities, where the RFC was identified as the best model and SVM as the second-best, the p-value of 0.0033 demonstrates a statistically significant difference between their performances. This strongly supports RFC's superior detection capabilities for Etherlock patterns. Similarly, for Block Dependency, RFC, identified as the best model over DT, showed a statistically significant difference with a p-value of 0.0286, reinforcing RFC's lead. in the challenging task of Reentrancy detection, the RFC's performance over DT (which was the second best) is highly statistically significant, indicated by a p-value of 0.0005. This further solidifies RFC's detection superiority especially given the inherent difficulties in detecting reentrancy.

In contrast, for Integer Underflow/Overflow vulnerabilities, while RFC was the top performer and DT the second best, the p-value of 0.1368 suggests that the performance difference between RFC and DT is not statistically significant at conventional alpha levels (e.g., 0.05). This indicates that for Integer Underflow/Overflow, both RFC and Decision Tree offer comparably robust and effective detection capabilities. The statistical testing in Table 11 affirms that RFC generally provides a significant and reliable advantage over other strong algorithms across most vulnerability types, validating its selection as the most robust model for bytecode analysis tasks.

Table 12 demonstrates a significant disparity in execution time between traditional static analysis tools and our machine learning model. While established tools like Slither and Smartcheck complete analysis in thousands of milliseconds (5,000 ms and 10,000 ms respectively), and more complex tools like Honeybadger, Maian, and Manticore range from tens of thousands to over a million milliseconds (e.g., Manticore at 1,468,000 ms), our model achieves an average execution time of 17.90 ms. This dramatic difference

in speed is primarily due to fundamental architectural differences. Traditional static analyzers often rely on computationally intensive techniques such as symbolic execution, abstract interpretation, or theorem proving, which involve exploring numerous execution paths and solving complex constraints to achieve high soundness and precision. In contrast, once our machine learning model is trained, its vulnerability detection process involves what is essentially an efficient rapid pattern matching operation. The average inference speed of a single model approximates to 0.58 milliseconds.

Table 13 presents a comparative analysis of vulnerability detection performance, showcasing True Positive (TP) and False Positive (FP) rates for various tools alongside the proposed model in this paper. A hyphen (-) indicates that information for that specific metric was not available for the respective tool. Our model consistently demonstrates high true positive rates across all vulnerability types: 99.5% for Etherlock (EL TP), 89% for Reentrancy (RE TP), 94.0% for Block Dependency (BD TP), and 98.0% for Integer Over/Underflow (IO/U TP). Concurrently, our model maintains relatively low false positive rates (EL FP: 0.08%, RE FP: 20.0%, BD FP: 12.7%, IO/U FP: 8.5%). When compared to other tools, while some, like Confuzzius and Contractfuzzer, show 100% TP for certain vulnerabilities, their reported FP rates are often incomplete or absent for direct comparison. Notably, our model achieves a strong balance of high detection rates with reasonably controlled false positives across all four distinct vulnerability categories, providing a more comprehensive and consistently evaluated performance profile compared to many existing tools. It should be noted that the test set of static analyzers range from only 2 to about 130 curated contracts, while our model is tested with a larger test set of hundreds of vulnerable contracts.

Table 14 presents a comparison of our model's performance metrics against other similar approaches in the literature in reentrancy detection. Reentrency is the common vulnerability between the models to compare with. Our model, utilizing a Random Forest Classifier, achieved an Accuracy of 0.790, Precision of 0.810, Recall of 0.850, and an F1-score of 0.820.

In comparison, the K-means Clustering approach [62] showed a Precision of 0.590, Recall of 0.530, and F1-score of 0.560. Another Random Forest Classifier approach [60] yielded an Accuracy of 0.333, Precision of 0.272, Recall of 0.467, and an F1-score of 0.343. Our model consistently reports higher metrics across Accuracy, Precision, Recall, and F1 when compared to both listed approaches. Moreover, when a combined dataset was tested for a multi-label SVM model, it achieved Accuracy, Recall, Precision, and F1 Score of 0.702, 0.833, 0.706, and 0.764 respectively. An ensemble approach allows for better results by separating the classification tasks, which reduces the error rate.

5.4 Performance efficiency

The presented stacked ensemble model demonstrates a strategic approach to balancing high detection accuracy with computational efficiency, a critical consideration in smart contract vulnerability analysis. Our methodology's core strength lies in its ability to effectively process a streamlined input: stripped hexadecimal opcode sequences. As detailed in Section 4.1.1, this preprocessing significantly reduces the data's dimensionality and complexity, allowing for more efficient training and inference compared to analyzing raw source code or highly verbose representations. Although advanced architectures such as large Graph Neural Networks (GNNs) and deep semantic analysis techniques may capture vulnerable code in high-level, they often demand substantial computational resources and face scalability challenges for comprehensive real-world applications [68]. Our ensemble model, in contrast, demonstrates high accuracy operating on this optimized input. This design avoids the singular, monolithic overhead associated with extremely large or deeply integrated single-architecture models. Furthermore, by building a model stack instead of a singular multi-label model, we achieve improved results by reducing class dimensionality. Consequently, our approach offers a highly effective and computationally manageable solution, proving to be more adaptable for large-scale smart contract vulnerability detection compared to resource-intensive static or symbolic analysis methods that often struggle with scalability. Fig. 3 below demonstrates the auditor tool where contracts are uploaded for analysis.

5.5 Model deployment to tool

The trained model is integrated into a standalone web-app tool. To achieve this, the model was serialized and saved using joblib, a module from the scikit-learn package in Python. Upon analysis, the tool outputs the contract address along with a label indicating whether the contract is vulnerable in a nearly instantaneous manner as demonstrated in Fig. 4. This integration enhances the practical applicability, making it suitable for deployment and use.

5.6 Limitations and future works

While our model demonstrates competitive performance in detecting smart contract vulnerabilities, a primary limitation of this approach, common in anomaly detection domains, is the inherent challenge of acquiring a sufficiently large and diverse dataset of real-world vulnerable contracts. Although

techniques like SMOTE (Synthetic Minority Over-sampling Technique) were employed to mitigate class imbalance within the dataset, the reliance on synthesized data for a substantial portion of the minority (vulnerable) class could potentially limit the model's ability to capture the full spectrum of nuanced, real-world vulnerability patterns. A larger, organically grown dataset of labeled vulnerable contracts would significantly enhance the model's generalizability and robustness, potentially leading to further improvements in detection accuracy and a more reliable representation of its performance on unseen, complex vulnerabilities.

Additionally, the accuracy of our model is fundamentally bound by the quality and comprehensiveness of the ground truth labels used for training; if these labels are derived from existing static analysis tools, the model can only learn to detect vulnerabilities that these tools are capable of identifying. We mitigate this through the conjunction of multiple tools and confidence weighing, however, this remains a concern, especially considering vulnerabilities that are inherently difficult to identify. With that in mind, we identify future directions for this research.

1. **Dataset Expansion and Diversity:** A critical direction involves expanding the dataset, particularly by acquiring a larger and more diverse collection of real-world vulnerable contracts. This would reduce the reliance on synthetic data generation techniques like SMOTE, leading to models that generalize more robustly to complex and unseen vulnerability patterns.
2. **Enhanced Feature Engineering and Contextual Analysis:** For vulnerabilities that proved challenging for opcode analysis alone, such as Reentrancy and Block Dependency, future work could focus on incorporating richer, more semantically aware features. This includes integrating information from control flow graphs (CFGs), data flow analysis, or abstract syntax trees (ASTs) in a manner that is model-trainable to capture intricate execution paths, inter-contract interactions, and variable dependencies that are not fully apparent from sequential opcodes.
3. **Ablation study on opcode pre-processing:** A deeper investigation into the relationship between opcode pre-processing (specifically, the stripping of opcode values) and detection performance. Such study would seek to answer the research question:"What is the quantifiable impact on detection performance when opcode values are not removed during pre-processing?". Moreover, do models overfit to specific opcode patterns more than others?

6 Conclusion

Blockchain is a disruptive technology that reimagines online transactions and asset ownership through crypto currencies and non-fungible tokens. While blockchain and through the use of smart contracts, provide a sea of advantages and convenience they still suffer from code vulnerabilities. Another major problem is the fact that smart contracts are immutable. This means once a vulnerability is discovered, a redirection to a new contract must be done or destruction of the old contract is necessary. Several vulnerabilities can be leveraged by attackers to ensue major damage, including Etherlock, Block Dependancy, Integer Overflow/Underflow and Reentrancy. In this paper, we propose a machine learning model stack to detect these vulnerabilities in smart contracts. Moreover, we integrate these models into a standalone tool that with an average execution speed of 17.90ms per contract. The proposed models achieved exceptionally in comparison with similiar works, F1 scores ranging from 82.0% - 99.9%, demonstrating the viability of machine-learning solutions and the enhanced performance of model stacking compared to traditional multi-label single classifiers.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s10586-025-05725-y.

Author contributions R.A: wrote the main manuscript, performed literature review, conceived the main contributions from the original manuscript, prepared figures, tables and listings. Designed and implemented weighing methodology. Development of Preprocessing code. Supported model tuning and dataset creation process. Performed analysis and discussion of results. Performed all citation works.
T.S: Programmed dataset collection code, programmed and worked on tuning of all models. Development of tool and interface. Testing and validation of results. Development of Preprocessing code. Supported converting manuscript to tex.
W.A: Supervision of the project and manuscript review
Q.H: Designed source contract to processable dataset method, supervision of the project and manuscript review

Data availability Project tool and source code developed in this manuscript has been uploaded to Github: https://github.com/Cosmic-Clank/Solidity-Vuln-Detection Dataset: https://www.kaggle.com/datasets/rha4365/vulnerable-solidity-smart-contracts-dataset/data

Declarations

Ethics declaration Not applicable.

Competing Interests The authors declare no competing interests.

Financial Statement No funds, grants, or other support was received.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the

source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Bitcoin, S. N.: A peer-to-peer electronic cash system, (2008). https://bitcoin.org/bitcoin.pdf
2. Wood, G.: Ethereum: A secure decentralised generalised transaction ledger. Ethereum Project Yellow Paper, vol. 151 (2014)
3. Arooj, A., Farooq, M.S., Umer, T.: Unfolding the blockchain era: Timeline, evolution, types and real-world applications. J. Netw. Comput. Appl. **207**, 103511 (2022)
4. Qatawneh, M., Almobaideen, W., Abualganam, O.: Challenges of blockchain technology in context internet of things: A survey. International Journal of Computer Applications **175**, 975–8887 (2020)
5. Alhasan, B., Qatawneh, M., Almobaideen, W.: Blockchain technology for preventing counterfeit in health insurance. In 2021 International Conference on Information Technology (ICIT), pages 935–941 (2021)
6. Almobaideen, W., Mohammed, M.: Vaxina: Decentralized vaccination tracking system. In 2023 9th International Conference on Information Technology Trends (ITT), pages 38–43 (2023)
7. Zheng, Z., Xie, S., Dai, H.-N., Chen, W., Chen, X., Weng, J., Imran, M.: An overview on smart contracts: Challenges, advances and platforms. Future Generation Computer Systems **105**, 475–491 (2020)
8. Lashkari, B., Musilek, P.: A comprehensive review of blockchain consensus mechanisms. IEEE Access **9**, 43620–43652 (2021)
9. Behnke, R.: Explained: The poly network hack, July 2023. Accessed: 2025-02-20 (2023)
10. Ponciano, J.: More than \$600 million stolen in ethereum and other cryptocurrencies, marking one of crypto's biggest hacks ever. Accessed: 2025-02-20 (2021)
11. Research, E.: The largest theft in history - following the money trail from the bybit hack (2025)
12. Almobaideen, W., Alnuman, R., Sajid, T., Hasan, Q.: Svm machine learning model for detection of etherlock vulnerability in solidity smart contracts. In 2024 6th International Conference on Blockchain Computing and Applications (BCCA), pages 434–440 (2024)
13. Seneviratne, O.: The feasibility of a smart contract "kill switch". In 2024 6th International Conference on Blockchain Computing and Applications (BCCA), pages 473–480 (2024)
14. Magalhães, J.: Current Vulnerabilities and Risks of Blockchain: A Systematic Literature Review, pp. 159–175. Springer Nature Switzerland, Cham (2025)
15. Jeon, S., Lee, G., Kim, H., Woo, S.S.: Design and evaluation of highly accurate smart contract code vulnerability detection framework. Data Min. Knowl. Disc. **38**(3), 888–912 (2024)
16. Almakhour, M., Sliman, L., Samhat, A.E., Mellouk, A.: A formal verification approach for composite smart contracts security using fsm. Journal of King Saud University **35**(1), 70–86 (2023)
17. Proskauer Rose LLP. When smart contracts are outsmarted: The parity wallet freeze and software liability in the internet of value, (2017)
18. Kushwaha, M., Mukherjee, A., Pandey, A., Halder, R.: Semantics-based static vulnerability detection in solidity using abstract interpretation. In Vishwas T. Patil, Ram Krishnan, and Rudrapatna K. Shyamasundar, editors, Information Systems Security, pages 265–285, Cham. Springer Nature Switzerland (2025)
19. Lai, E., Luo, W.: Static analysis of integer overflow of smart contracts in ethereum. In Proceedings of the 2020 4th International Conference on Cryptography, Security and Privacy, ICCSP 2020, page 110-115, New York, NY, USA. Association for Computing Machinery (2020)
20. Dong, C., Li, Y., Tan, L.: A new approach to prevent reentrant attack in solidity smart contracts. In Xueming Si, Hai Jin, Yi Sun, Jianming Zhu, Liehuang Zhu, Xianhua Song, and Zeguang Lu, editors, Blockchain Technology and Application, pages 83–103, Singapore. Springer Singapore (2020)
21. Abdelaziz, M.T., Radwan, A., Mamdouh, H., Saad, A.S., Abuzaid, A.S., AbdElhakeem, A.A., Zakzouk, S., Moussa, K., Saeed Darweesh, M.: Enhancing network threat detection with random forest-based nids and permutation feature importance. J. Netw. Syst. Manage. **33**(1), 2 (2024)
22. Ghaffari, A., Jelodari, N., Pouralish, S., Derakhshanfard, N., Arasteh, B.: Securing internet of things using machine and deep learning methods: A survey. Clust. Comput. **27**(7), 9065–9089 (2024)
23. Alnuman, R., Hyder, S. I., Ovaz Akpinar, K.: Hvac attack detection using novel machine learning model. In 2023 9th International Conference on Information Technology Trends (ITT), pages 14–19, (2023)
24. AlSobeh, A.M.R., Gaber, K., Hammad, M.M., Nuser, M., Shatnawi, A.: Android malware detection using time-aware machine learning approach. Cluster Computing **27**(9), 12627–12648 (2024)
25. Farfoura, M.E., Mashal, I., Alkhatib, A., Batyha, R.M.: A lightweight machine learning methods for malware classification. Clust. Comput. **28**(1), 1 (2024)
26. Tang, Z., Hu, Q., Hu, Y., Kuang, W., Chen, J.: Sevuldet: A semantics-enhanced learnable vulnerability detector. In 2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pages 150–162 (2022)
27. Nguyen, V., Nepal, S., Wu, T., Yuan, X., Rudolph, C.: Safe: Advancing large language models in leveraging semantic and syntactic relationships for software vulnerability detection, (2024)
28. Hearst, M.A., Dumais, S.T., Osuna, E., Platt, J., Scholkopf, B.: Support vector machines. IEEE Intelligent Systems and their Applications **13**(4), 18–28 (1998)
29. Parmar, A., Katariya, R., Patel, V.: A review on random forest: An ensemble classifier. In Jude Hemanth, Xavier Fernando, Pavel Lafata, and Zubair Baig, editors, International Conference on Intelligent Data Communication Technologies and Internet of Things (ICICI) 2018, pages 758–763, Cham. Springer International Publishing (2019)
30. Song, Y.-Y., Ying, L.: Decision tree methods: applications for classification and prediction. Shanghai Arch. Psychiatry **27**(2), 130–135 (2015)
31. Halder, R.K., Uddin, M.N., Ashraf Uddin, Md.., Aryal, S., Khraisat, A.: Enhancing k-nearest neighbor algorithm: a comprehensive review and performance analysis of modifications. Journal of Big Data **11**(1), 113 (2024)
32. Yong, Yu., Si, X., Changhua, H., Zhang, J.: A review of recurrent neural networks: Lstm cells and network architectures. Neural Comput. **31**(7), 1235–1270 (2019)
33. Li, Z., Liu, F., Yang, W., Peng, S., Zhou, J.: A survey of convolutional neural networks: Analysis, applications, and prospects. IEEE Transactions on Neural Networks and Learning Systems **33**(12), 6999–7019 (2022)

34. Liao, J.-W., Tsai, T.-T., He, C.-K., Tien, C.-W.: Soliaudit: Smart contract vulnerability assessment based on machine learning and fuzz testing. In 2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS), pages 458–465, (2019)
35. Krichen, M.: Strengthening the security of smart contracts through the power of artificial intelligence. Computers **12**(5), 107 (2023)
36. Colin, L.S.H., Mohan, P.M., Pan, J., Keong, P.L.K.: An integrated smart contract vulnerability detection tool using multi-layer perceptron on real-time solidity smart contracts. IEEE Access **12**, 23549–23567 (2024)
37. Momeni, P., Wang, Y., Samavi, R.: Machine learning model for smart contracts security analysis. In 2019 17th International Conference on Privacy, Security and Trust (PST), pages 1–6, (2019)
38. Zhen, Z., Zhao, X., Zhang, J., Wang, Y., Chen, H.: Da-gnn: A smart contract vulnerability detection method based on dual attention graph neural network. Comput. Netw. **242**, 110238 (2024)
39. Gu, W., Wang, G., Li, P., Li, X., Zhai, G., Li, X., Chen, M.: Detecting unknown vulnerabilities in smart contracts with multi-label classification model using cnn-bilstm. In Guojun Wang, Kim-Kwang Raymond Choo, Jie Wu, and Ernesto Damiani, editors, Ubiquitous Security, pages 52–63. Springer Nature Singapore, (2023)
40. Tang, X., Yuying, D., Lai, A., Zhang, Z., Shi, L.: Deep learning-based solution for smart contract vulnerabilities detection. Sci. Rep. **13**(1), 20106 (2023)
41. Deng, W., Wei, H., Huang, T., Cao, C., Peng, Y., Hu, X.: Smart contract vulnerability detection based on deep learning and multimodal decision fusion. Sensors **23**(16), 7246 (2023)
42. BEKTAS, B. C.: Vulnerability Detection on Solidity Smart Contracts by Using Convolutional Neural Networks. PhD thesis, Middle East Technical University, (2023)
43. Brent, L., Jurisevic, A., Kong, M., Liu, E., Gauthier, F., Gramoli, V., Holz, R.: and Bernhard Scholz. A scalable security analysis framework for smart contracts, Vandal (2018)
44. Li, P., Wang, G., Xing, X., Li, X., Zhu, J.: Detecting unknown vulnerabilities in smart contracts using opcode sequences. Connect. Sci. **36**(1), 2313853 (2024)
45. Trail of Bits. Slither: Static analyzer for solidity and vyper. https://github.com/crytic/slither, (2018)
46. Mythril, C.: A security analysis tool for evm bytecode. https://github.com/ConsenSys/mythril, (2017)
47. Feist, J., Grieco, G., Groce, A.: Slither: A static analysis framework for smart contracts. In 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), page 8-15. IEEE (2019)
48. Luu, L.: Oyente: An analysis tool for smart contracts. https://github.com/enzymefinance/oyente, (2016)
49. Luu, L., Chu, D.-H., Olickel, H., Saxena, P., Hobor, A.: Making smart contracts smarter. In Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS '16, page 254-269, New York, NY, USA, 2016. Association for Computing Machinery
50. Tsankov, P., Sachinoglou, Y., Nuckols, C., and iczc. Securify v2.0. https://github.com/eth-sri/securify2, (2020)
51. Tsankov, P., Dan, A., Drachsler-Cohen, D., Gervais, A., Bünzli, F., Vechev, M.: Securify: Practical security analysis of smart contracts. CCS '18, page 67-82, New York, NY, USA. Association for Computing Machinery (2018)
52. Nikolic, I., Kolluri, A., Sergey, I., Saxena, P., Hobor, A.: Maian: Automatic tool for finding trace vulnerabilities in ethereum smart contracts. https://github.com/ivicanikolicsg/MAIAN, (2018)
53. Nikolic, I., Kolluri, A., Sergey, I., Saxena, P., Hobor, A.: Finding the greedy, prodigal, and suicidal contracts at scale, (2018)
54. Jiang, B., Liu, Y., Chan, W. K.: Contractfuzzer: The ethereum smart contract fuzzer for security vulnerability detection. https://github.com/gongbell/ContractFuzzer, (2018)
55. Conkas, N. V.: A modular static analysis tool for ethereum bytecode. https://github.com/nveloso/conkas, (2019)
56. Conkas, N. V.: A modular and static analysis tool for ethereum bytecode. https://fenix.tecnico.ulisboa.pt/downloadFile/1689244997262417/94080-Nuno-Veloso_resumo.pdf. Bachelor's thesis (2021)
57. Torres, C. F., Iannillo, A. K., Gervais, A., State, R.: Confuzzius: A data dependency-aware hybrid fuzzer for ethereum smart contracts. https://github.com/christoftorres/ConFuzzius, (2021)
58. Torres, C. F., Iannillo, A. K., Gervais, A., State, R.: Confuzzius: A data dependency-aware hybrid fuzzer for smart contracts. In 2021 IEEE European Symposium on Security and Privacy (EuroSP),103–119, (2021)
59. Codes, E.V.M.: Contributors. Evm opcodes reference. https://www.evm.codes
60. Zou, H., Li, Z., Li, X.: Malicious code detection in smart contracts via opcode vectorization, (2025)
61. Bresil, M., Prasad, P.W.C., Sayeed, M.S., Bukar, U.A.: Deep learning-based vulnerability detection solutions in smart contracts: A comparative and meta-analysis of existing approaches. IEEE Access **13**, 28894–28919 (2025)
62. Gu, X., Yang, H., Liu, S., Cui, Z.: Smart contract vulnerability detection based on clustering opcode instructions. In The 35th International Conference on Software Engineering and Knowledge Engineering, SEKE 2023, KSIR Virtual Conference Center, USA, July 1-10, 2023. KSI Research Inc., (2023)
63. Gao, G., Li, Z., Jin, L., Liu, C., Li, J., Meng, X.: Rtms: A smart contract vulnerability detection method based on feature fusion and vulnerability correlations. Electronics **14**(4), 768 (2025)
64. Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P.: Empirical review of automated analysis tools on 47,587 ethereum smart contracts. In Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, ICSE '20, page 530-541. ACM, (2020)
65. Durieux, T., Abreu, R., Abreu, R.: Smartbugs: A framework to analyze smart contracts. https://github.com/smartbugs/smartbugs-wild. Accessed: 2025-05-28 (2020)
66. Rincy, T.N., Gupta, R.: Ensemble learning techniques and its efficiency in machine learning: A survey. In 2nd International Conference on Data, Engineering and Applications (IDEA), pages 1–6, (2020)
67. Jiang, B., Liu, Y., Chan, W.K.: Contractfuzzer: Fuzzing smart contracts for vulnerability detection. In 2018 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE), pages 259–269, (2018)
68. Zheng, Y., Yi, L., Wei, Z.: A survey of dynamic graph neural networks. Frontiers of Computer Science **19**(6), 196323 (2024)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.